

KI in der Produktion

Projekt: 3D Keypoint Detection

Gruppe 2:

1.1 Projektfokus



Aufgabenstellung

- Erkennung von Keypoints an 3D-Objekten
- Objekte werden als Punktwolken repräsentiert → Datensatz
- Untersuchung modellbasierter Verfahren zur Keypoint-Detection

Ziel des Projekts

- Aufbau einer vollständigen Trainings- und Evaluationspipeline
- Training eines Modells auf Punktwolken
- Visualisierung der erkannten Keypoints
- Bewertung der Ergebnisse anhand von Rekonstruktion und Keypoint-Verteilung

→ **automatische Erkennung charakteristischer 3D-Keypoints auf punktwolkenbasierten Objektmodellen**

→ **Grundlage für Anwendungen wie Pose Detection, Visual Servoing und Object Classification**

1.2 Stand der Technik & Modellauswahl



Modell	Grundidee	Wie entstehen Keypoints?	Rekonstruktion über
UKPGAN	Keypoints als Informationskompression	Encoder + GAN-Mechanismen	Keypoints
SkeletonMerger	Keypoints bilden ein Skelett	Encoder sagt Keypoints und Kanten vorher	Skelett + lokale Punktwolken
KeyGrid	Keypoints erzeugen räumliche Heatmap	Encoder sagt Keypoints vorher	3D-Grid-Heatmap

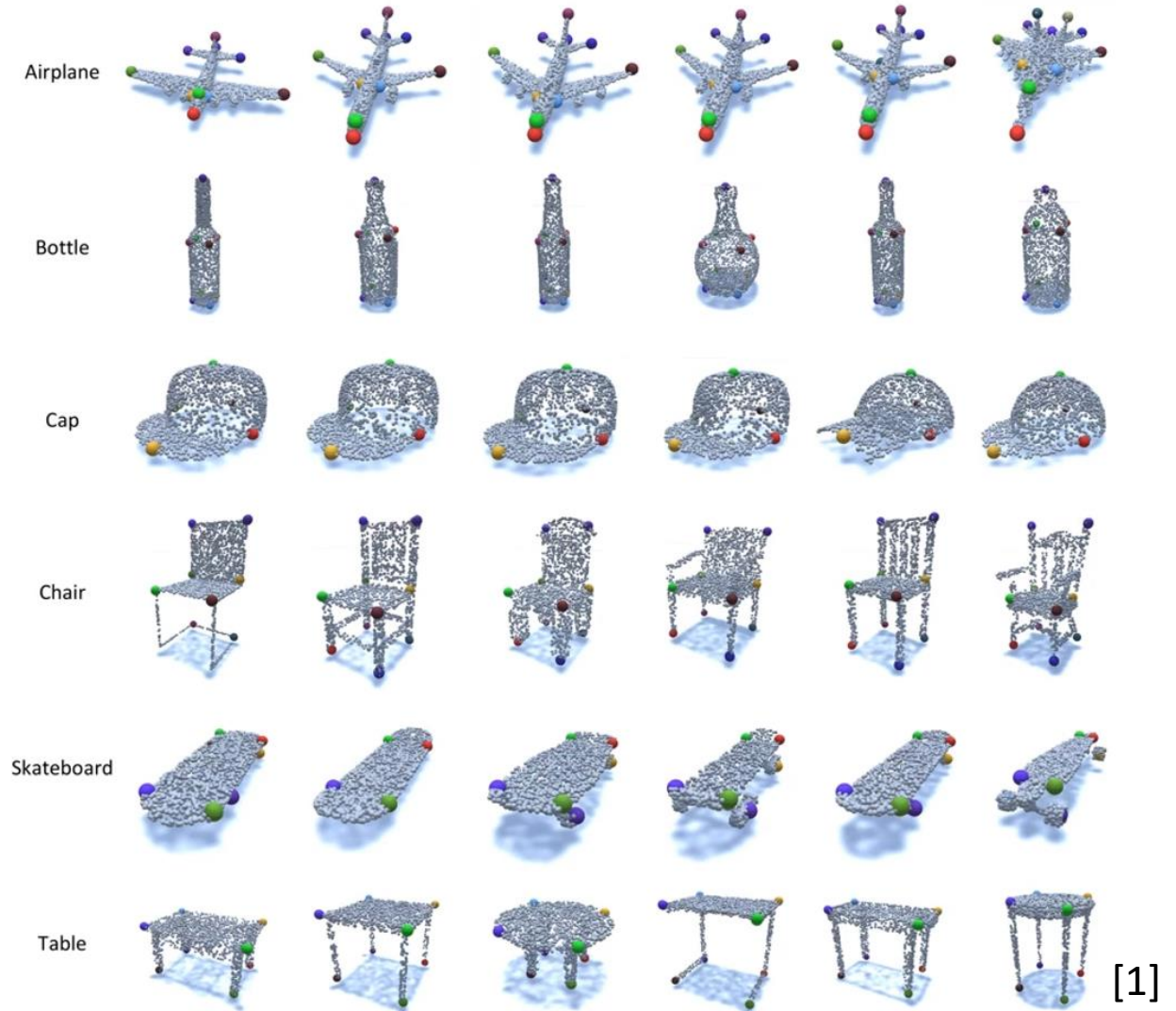
- Gleiches Ziel: aussagekräftige 3D-Keypoints lernen
- Unterschied: Wie stark werden Keypoints in die Rekonstruktion eingebunden?
- Modellwahl: KeyGrid koppelt Keypoints über eine 3D-Heatmap direkt an den Decoder

Gewählter Ansatz:
KeyGrid als alternatives Modell

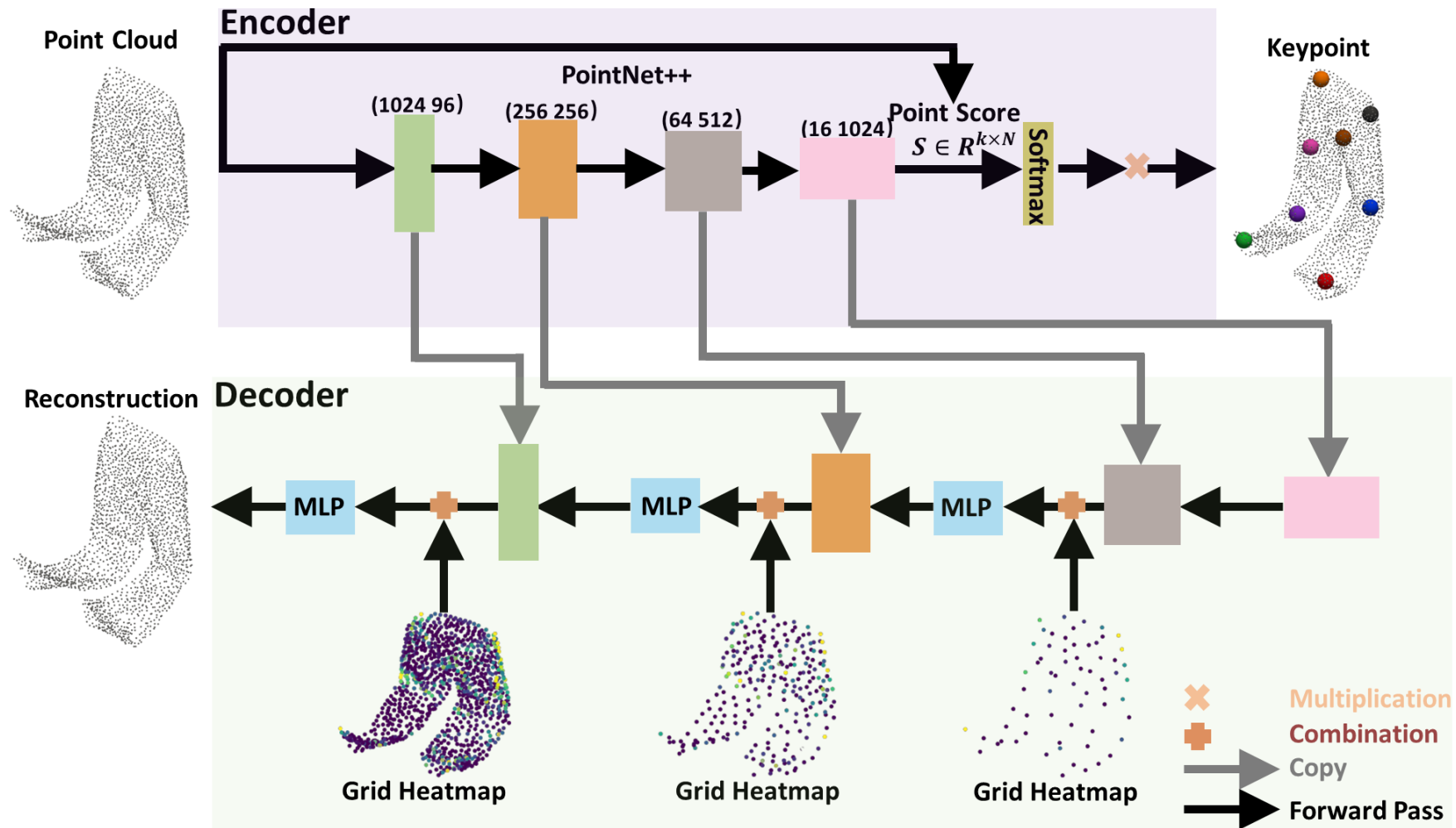
1.3 KeypointNet-Datensatz



Kategorie	Anzahl Modelle / Dateien
Airplane	1022
Bathtub	492
Bed	146
Bottle	380
Cap	38
Car	1002
Chair	999
Guitar	697
Helmet	90
Knife	270
Laptop	439
Motorcycle	298
Mug	198
Skateboard	141
Table	1124
Vessel	910
Gesamt	8329



2.1 Architektur des Autoencoders



Encoder-Decoder (PointNet++) mit Keypoint-Abzweig und Grid-Heatmap-Rückführung in den Decoder [2]

2.2 Verwendung einer Heatmap



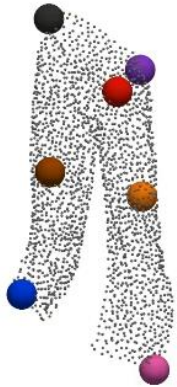
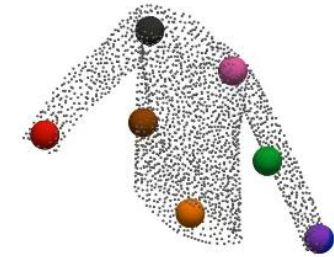
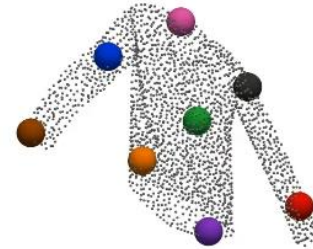
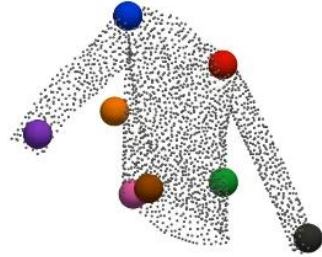
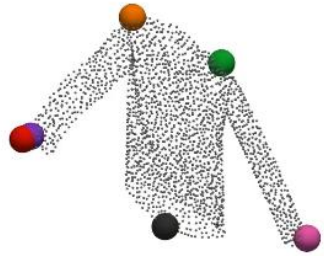
Was die Grid-Heatmap leistet

- 3D-Gitter über den Raum; pro Zelle Gauß-Wert nach Abstand zum nächsten Keypoint
- Heiß nahe Keypoint, kalt weit weg
- → glattes Wärmefeld
- **Einzige Brücke der Keypoints zum Decoder**
- Macht aus dem Autoencoder einen Keypoint-Detektor

Warum das entscheidend ist

- Ohne Heatmap ignoriert die Rekonstruktion die Keypoints → kein Lernsignal
- Schlechte Keypoints → schlechte Rekonstruktion → Training korrigiert sie
- Gitter statt roher Koordinaten = glatte Gradienten, stabiles Training
- Geometrische Verankerung → Keypoints konsistent auch bei Verformung (z.B. faltende Kleidung)

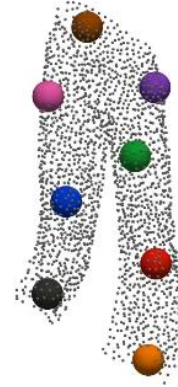
2.3 Vorteil einer Heatmap



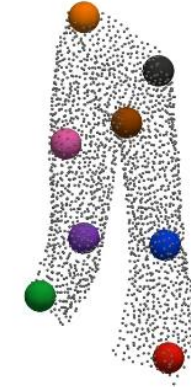
KD



SM



SC3K



Key-Grid

2.3 Verwendete Loss-Functions



Chamfer-Distance-Loss

- Vergleicht Rekonstruktion mit Original-Punktwolke
- Je Punkt: Abstand zum nächsten Punkt der anderen Wolke, in beide Richtungen
- Haupt-Lernsignal des Autoencoders
- $L = \sum \min \|x-y\|^2 (x \rightarrow y) + \sum \min \|y-x\|^2 (y \rightarrow x)$

Farthest-Point-Loss

- Regularisierer auf den k Keypoints
- Bestraft Keypoints, die zu nah beieinander liegen
- Erzwingt gute räumliche Verteilung über das Objekt
- Verhindert Kollaps aller Keypoints auf einen Fleck
- Gesamt: $L = L_{chamfer} + \lambda \cdot L_{fps}$

2.5 Training, Evaluation und Inferenz



Training

- Unsupervised – keine Keypoint-Labels
- Lernsignal nur aus Rekonstruktion
- Backprop durch Softmax, Gauß-Heatmap, Interpolation
- Schwierig: Keypoint-Kollaps, instabile frühe Epochen

Evaluation

- **Datensplit 70 / 15 / 15**
Train 70 % · Val 15 % · Test 15 %
- Metrik: Keypoint-Konsistenz über Instanzen
- (Prüfung auf Stabilität bei Verformung)

Inferenz

- Nur Encoder + Keypoint-Abzweig nötig
- Decoder/Heatmap nur fürs Training
- Ausgabe: k Keypoints je Punktwolke
- Voraussetzung: normierte, zentrierte Wolke

3.1 Setup & Voraussetzungen



Software & Frameworks

- Python 3.10
- PyTorch 2.5.1 + CUDA 11.8
- Conda-Umgebung (keygrid)
- PointNet++ / Key-Grid Codebasis
- Weights & Biases (optional, Logging)

Hardware & Voraussetzungen

- NVIDIA GeForce RTX 2070
- CUDA-fähige GPU zwingend
- Windows-Maschine
- KeypointNet-Datensatz lokal
- Ausreichend RAM für Punktwolken-Batches

3.2 Skripte und Datensatz



Datensatz: KeypointNet

- Großer annotierter 3D-Keypoint-Benchmark · Punktwolken über viele Objektkategorien (z.B. Flugzeug, Stuhl, Mensch) · menschlich gelabelte, konsistente Keypoints als Referenz

train.py

- Trainiert Autoencoder + Keypoints
- Chamfer- & FPS-Loss, Checkpoints

vision.py

- Evaluation auf Test-Split
- Metriken + Logging

visualization.py

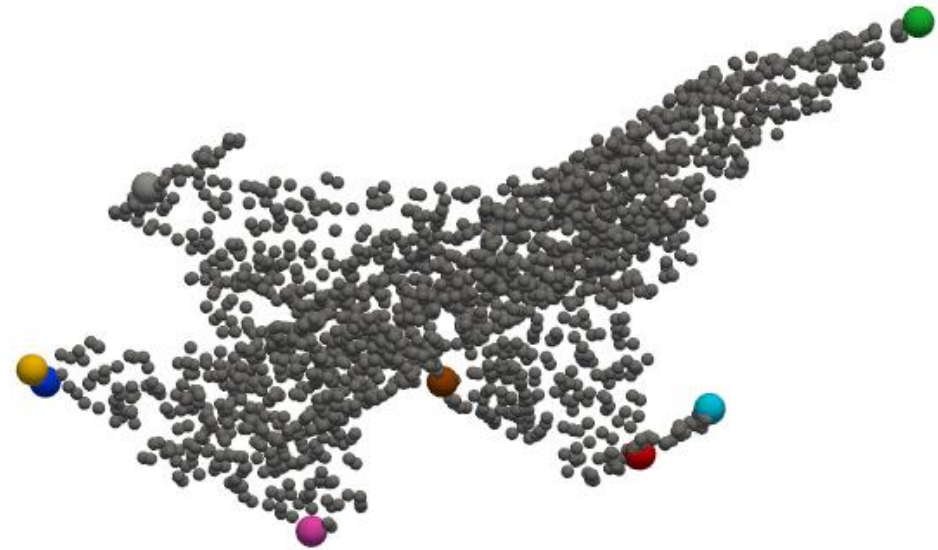
- Keypoints auf Punktwolke rendern
- Visueller Vergleich der Modelle

3.4 Trainingsbedingungen

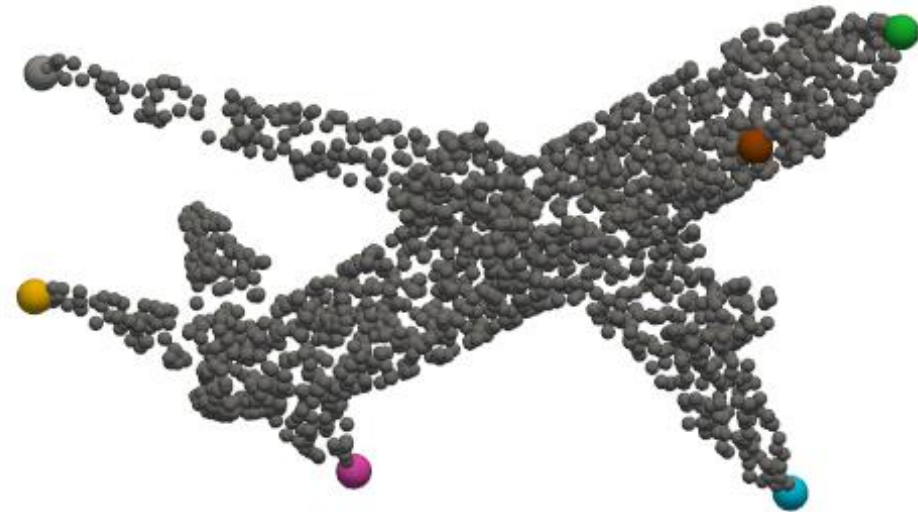
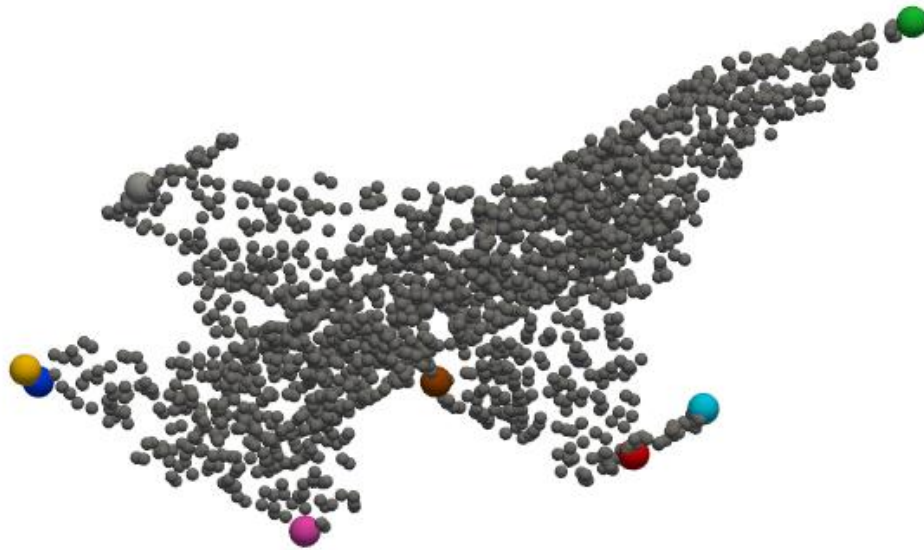


Vorgehen

- Training anhand von Keypoint-Menge
- Modelle **10** Keypoints
- Batchsize (16), Lernrate ($1 \cdot 10^{-3}$), Epochen(150) konstant
- Verwendeter Datensatz: Airplains



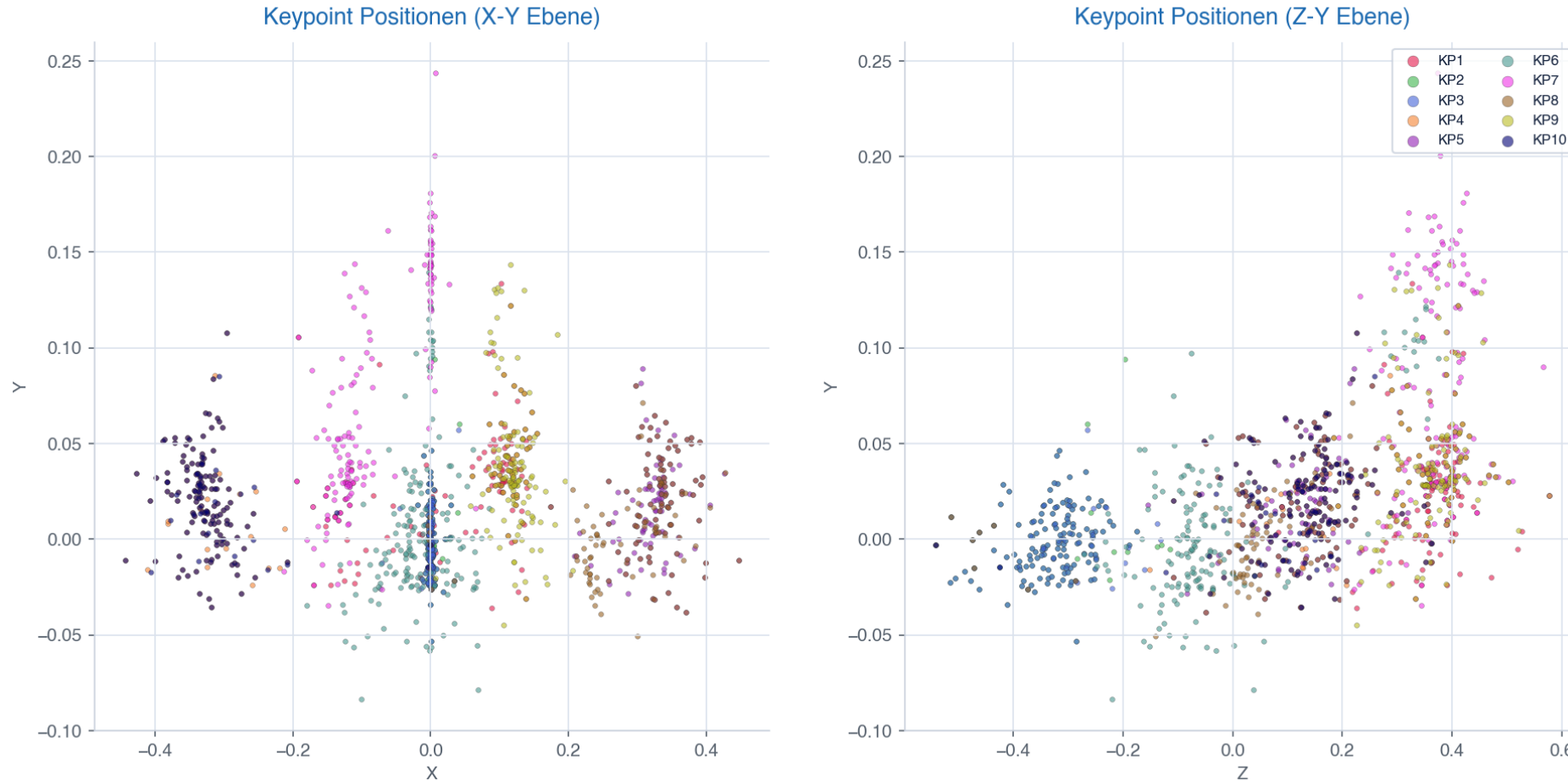
4.1 Ergebnis und Diskussion



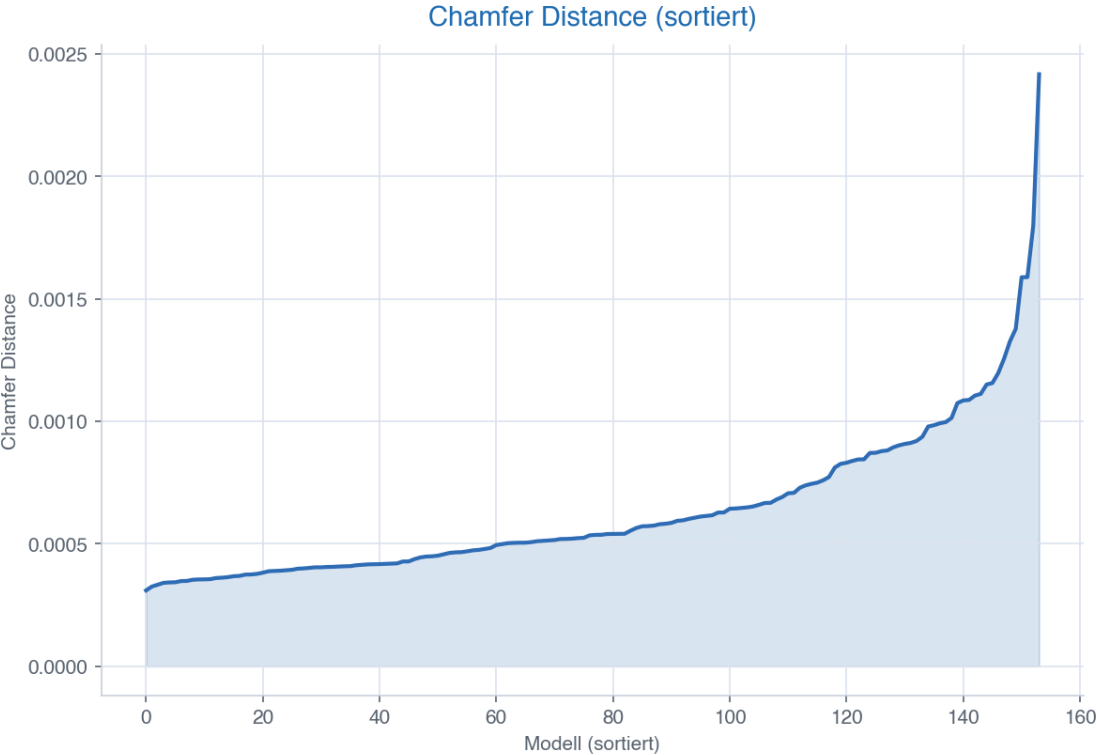
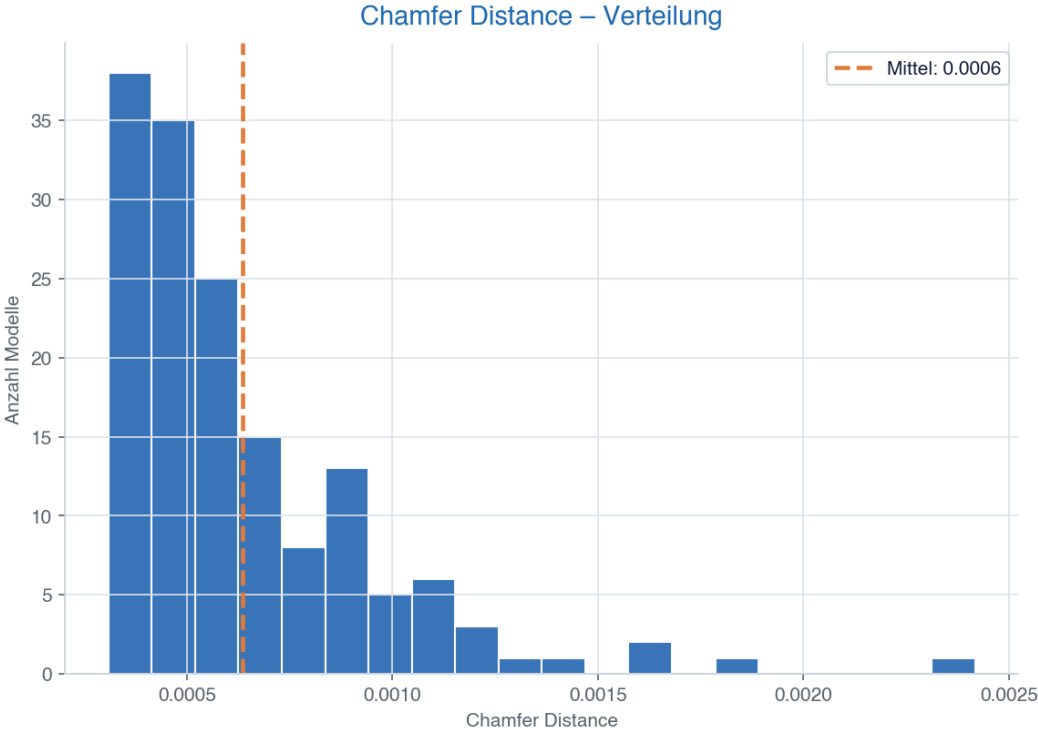
4.1 Ergebnis und Diskussion



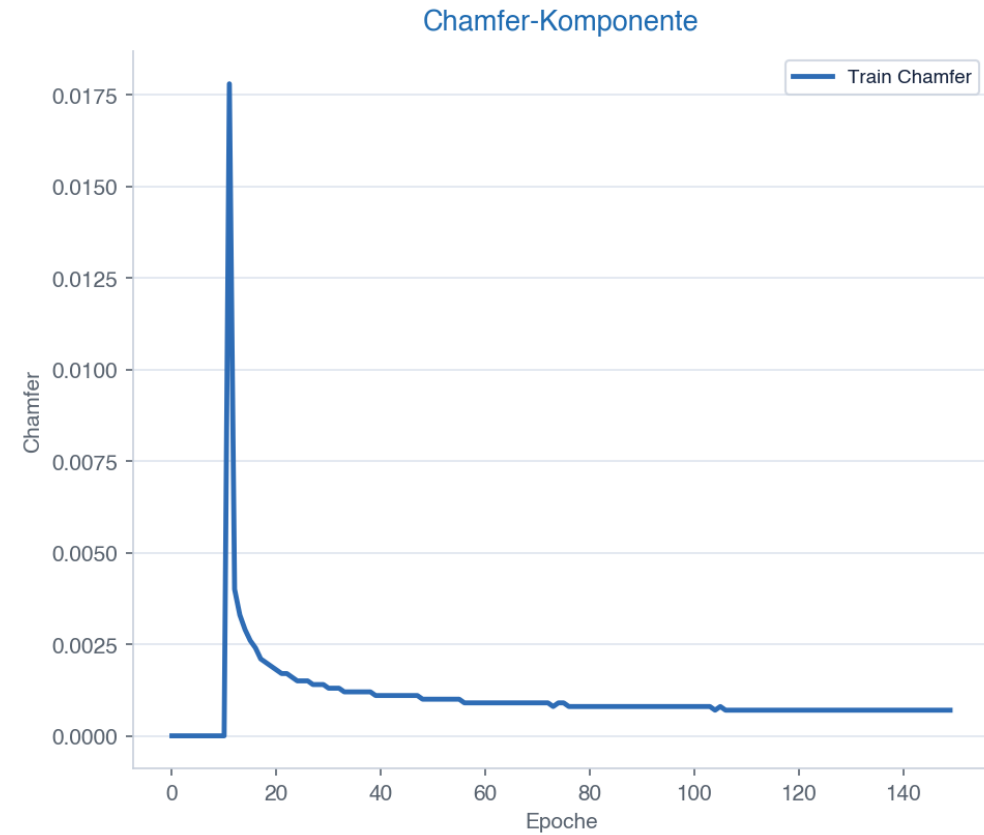
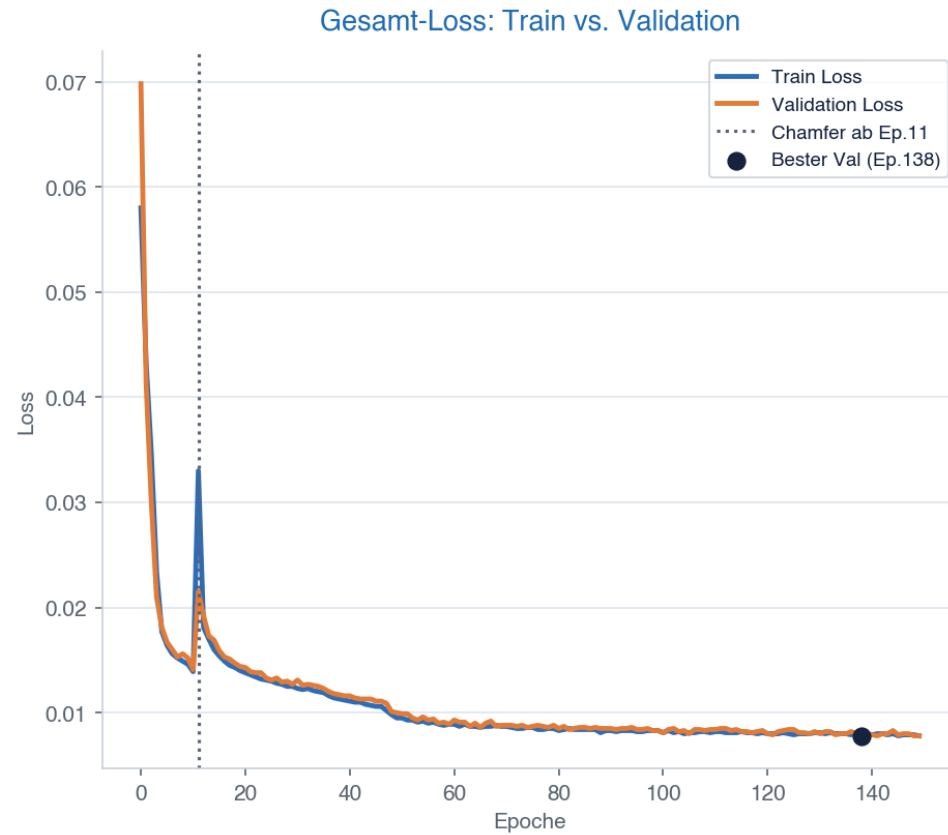
airplane_v3 (10 KP) – Keypoint-Verteilung



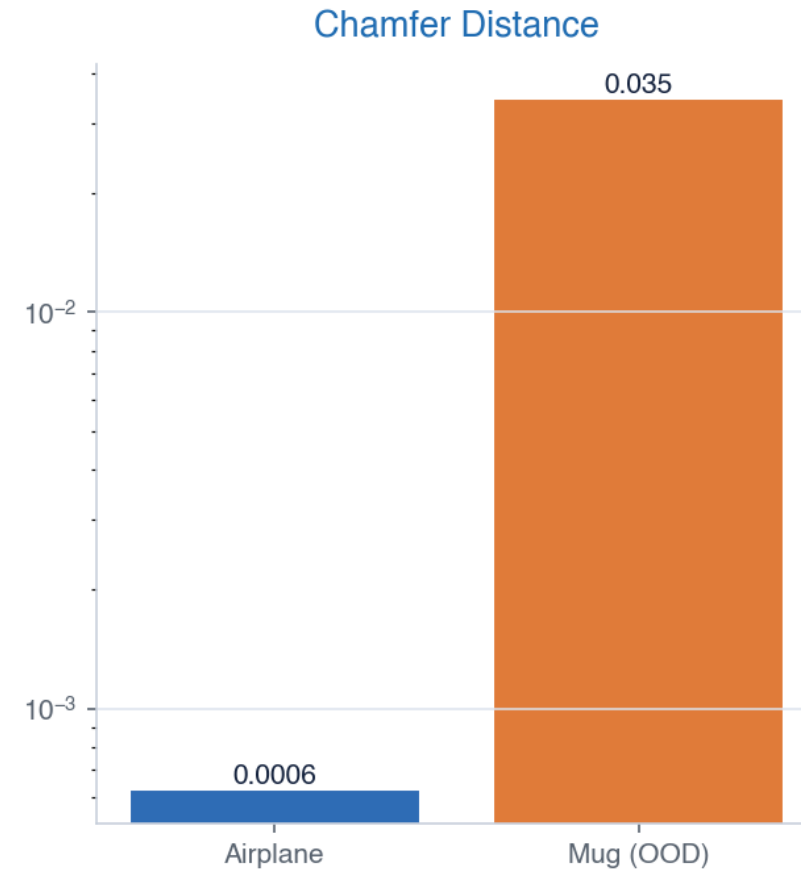
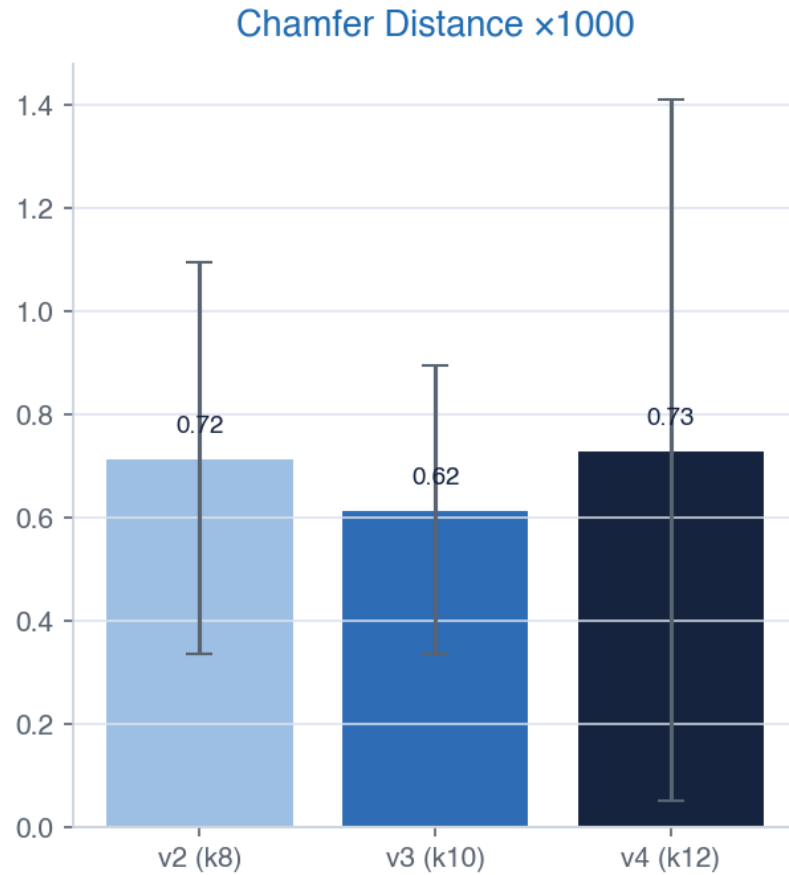
4.1 Ergebnis und Diskussion



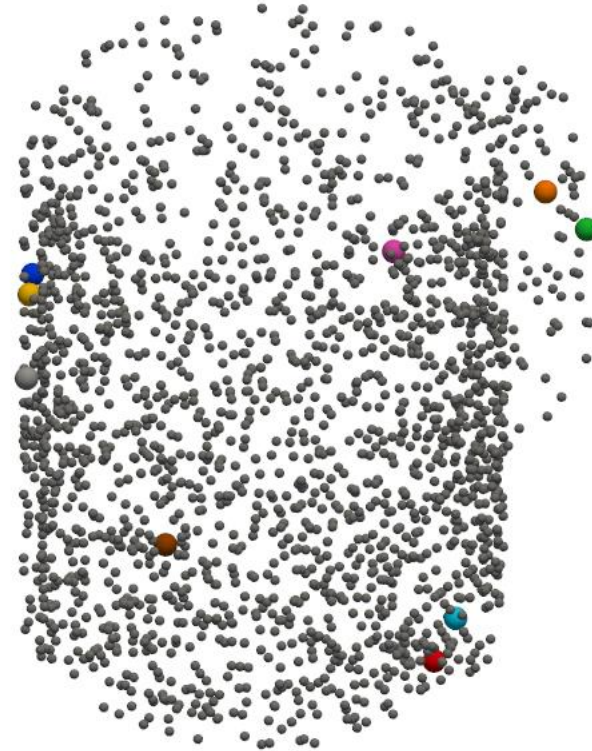
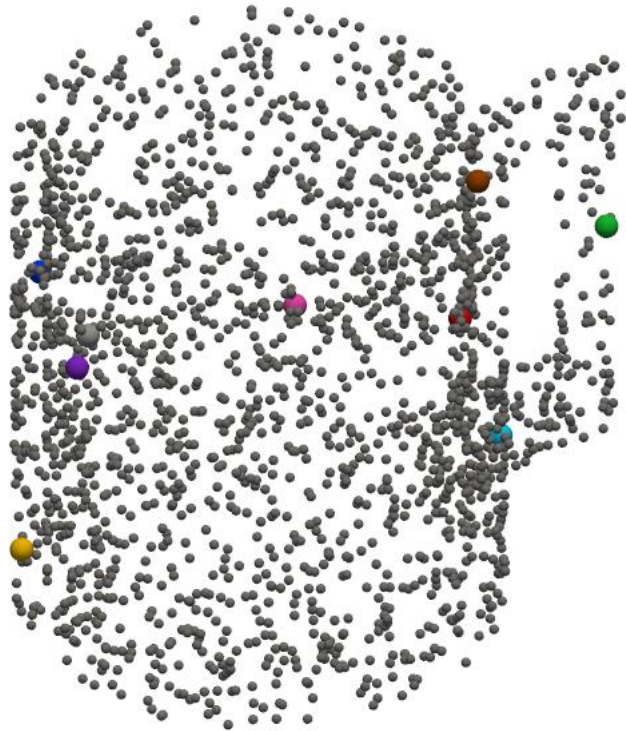
4.1 Ergebnis und Diskussion



4.2 Problemstellungen



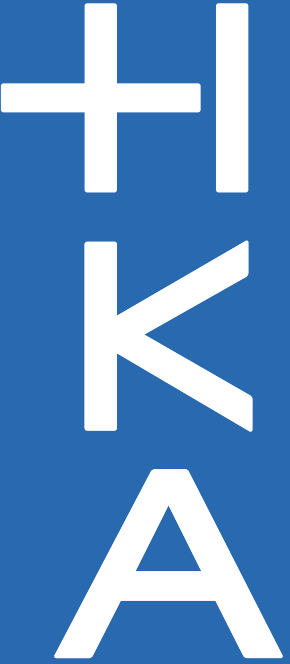
4.2 Problemstellungen





- [1] Y. You *et al.*, 'KeypointNet: A Large-scale 3D Keypoint Dataset Aggregated from Numerous Human Annotations', Aug. 07, 2020, *arXiv*: arXiv:2002.12687. doi: 10.48550/arXiv.2002.12687.
- [2] Key-Grid: Unsupervised 3D Keypoints Detection using Grid Heatmap Features'. Accessed: Jun. 30, 2026. [Online]. Available: <https://jackhck.github.io/keygrid.github.io/>
- [3] C. Hou, Z. Xue, B. Zhou, J. Ke, L. Shao, and H. Xu, 'Key-Grid: Unsupervised 3D Keypoints Detection using Grid Heatmap Features', Dec. 07, 2024, *arXiv*: arXiv:2410.02237. doi: 10.48550/arXiv.2410.02237.

Danke für Ihre Aufmerksamkeit!



Modellvergleich – Paper-Metriken

